Internet of Things (IoT) with Applications

Undergraduate Engineering Course

Module 1

Introduction to IoT

Foundations, Architecture and Devices

Unit I: Introduction to IoT

Concepts

Topics Covered:

Definition & Characteristics · Evolution of IoT · IoT vs Internet/M2M/CPS
Components & Enabling Technologies · Reference Architectures
IoT Devices · The IoT Data Journey

1 Introduction to IoT

The **Internet of Things (IoT)** refers to a network of physical objects – devices, vehicles, appliances, and other everyday items – embedded with sensors, software, and communication hardware that allows them to collect data from their environment, exchange that data over a network, and, in many cases, act back upon the physical world without requiring constant human operation. Where the traditional Internet connected people and computers, IoT extends connectivity to the physical world itself: a soil-moisture sensor in a field, a heart-rate monitor on a wrist, or a vibration sensor on a factory motor can all become active participants on a network, reporting their state and, where appropriate, receiving instructions in return.

This module develops the foundational vocabulary and reference models used throughout the course: what distinguishes IoT from related fields such as M2M communication and cyber-physical systems, what an IoT system is built from, how such systems are commonly layered into reference architectures, and how raw sensor readings ultimately become the dashboards and automated actions that make IoT systems useful.

1.1 Defining Characteristics of IoT

While no single definition is universally agreed upon, most treatments of IoT converge on a common set of defining characteristics:

- **Interconnection:** Devices are connected to each other and to the broader Internet, directly or through a gateway, enabling data to flow from the point of sensing to the point of use.
- **Sensing and actuation:** IoT devices typically combine the ability to sense some physical quantity (temperature, motion, location) with the ability, in many applications, to act on the physical world (opening a valve, switching a relay).
- **Heterogeneity:** An IoT deployment routinely mixes devices from many manufacturers, running different software, over different communication technologies – the system must tolerate and manage this diversity rather than assume a single uniform device type.
- **Scale:** Practical IoT deployments range from a handful of devices in a smart home to millions of devices across a national utility grid, so architectures must be designed to scale gracefully in both directions.
- **Autonomy and intelligence:** Increasingly, IoT systems are expected to operate with minimal human intervention, using embedded or cloud-based intelligence to interpret sensor data and trigger appropriate responses.
- **Context-awareness:** Many IoT applications derive their value precisely from knowing *where*, *when*, and *under what conditions* a reading was taken, rather than from the raw reading alone.

1.2 Evolution of IoT

The idea of networked, sensing objects has developed gradually over several decades, moving from simple remote telemetry toward today's AI-enabled, edge-connected systems. Table 1 sketches this progression in five broad eras; the boundaries between eras are not sharp, but the trend – from isolated connectivity toward autonomous intelligence – is consistent across the literature.

Era	Approximate Period	Defining Trait
Pre-IoT telemetry	1980s–1990s	Isolated remote monitoring/SCADA systems; no shared Internet connectivity.
Early IoT	Late 1990s–2000s	RFID tagging and the coining of the term "Internet of Things"; barcode-style identification extended to networked tags.
Connected IoT	2010s	Cheap Wi-Fi/Bluetooth modules and smartphones drive consumer and industrial device connectivity at scale.
Cloud-Integrated IoT	Mid-2010s onward	Devices stream data to cloud platforms for storage, analytics, and remote dashboards; the "IoT platform" emerges as a product category.
AI-Enabled / Edge IoT	2020s onward	Machine learning is pushed to the edge for real-time inference; 5G and edge computing reduce latency for time-critical applications.

Table 1: Five eras in the evolution of IoT.

1.3 IoT versus Related Fields

IoT is frequently confused with, or used interchangeably alongside, three related concepts. Distinguishing them precisely matters because each implies different design constraints.

- **IoT vs the Internet:** The Internet is a network of interconnected *computers* designed primarily to move data between people and services. IoT extends this network to *physical objects*, many of which have no user interface, limited power, and constrained processing capability – requirements the original Internet architecture was not designed around.
- **IoT vs M2M (Machine-to-Machine):** M2M communication predates IoT and refers narrowly to direct communication between two machines, typically over a dedicated, point-to-point or proprietary link (for example, a vending machine reporting stock levels over a cellular modem). IoT generalises this idea: devices communicate over shared, Internet-based infrastructure, are typically addressable individually, and participate in a broader ecosystem of cloud services, analytics, and other devices rather than a single fixed counterpart.
- **IoT vs Cyber-Physical Systems (CPS):** A CPS is a tighter integration of computation with physical processes, typically featuring a real-time feedback loop between sensing, computation, and actuation (an autonomous vehicle's control system is a CPS). IoT is broader and network-centric, and does not require the same real-time control guarantees; in practice, many CPS are built *using* IoT connectivity, but not every IoT deployment (e.g., a simple temperature logger) constitutes a CPS.

Worked Example

Classifying a system. A vending machine calls a single central server once per day over a cellular modem to report its stock levels, using a fixed, manufacturer-proprietary protocol; a smart thermostat continuously reports temperature to a cloud platform, receives software updates over Wi-Fi, and can be controlled from a phone app anywhere in the world. Which system is best described as IoT, and which is better described as classic M2M?

Aspect	Internet	M2M	CPS
Primary link	Human/computer to computer	Machine to machine, often point-to-point	Sensing–computation–actuation loop
Networking	Shared, open, Internet Protocol	Often proprietary or dedicated	May use IoT or dedicated links
Real-time need	Not inherent	Low to moderate	Typically high, safety-critical
Scale	Global	Small, fixed groups	Localised, tightly coupled

Table 2: IoT's relationship to the Internet, M2M, and CPS.

Solution. The vending machine's proprietary, point-to-point link to one fixed server, with no broader network participation, is characteristic of classic M2M. The thermostat's use of shared Internet infrastructure, cloud integration, remote accessibility, and participation in a wider platform ecosystem is characteristic of IoT.

2 Components and Enabling Technologies

Every IoT system, regardless of application domain, can be decomposed into a small set of recurring building blocks. Understanding these components independently of any one application is what allows the reference architectures of Section 3 to apply uniformly across smart agriculture, healthcare, industry, and smart cities alike.

Component	Role
Sensors	Convert a physical quantity (temperature, light, motion, gas concentration) into an electrical signal that can be read digitally.
Actuators	Convert a digital command back into a physical action (opening a valve, switching a relay, moving a motor).
Microcontrollers / Embedded Systems	Provide local computation, read sensors, drive actuators, and manage communication; examples include Arduino, ESP32, and Raspberry Pi.
Connectivity	Short- or long-range communication technologies that move data between devices, gateways, and the cloud (Section 4 of Unit II covers these in depth).
Gateway	Aggregates data from many local devices, performs protocol translation, and forwards data to the cloud, often adding a first layer of filtering or edge processing.
Cloud / Processing Platform	Stores data at scale, runs analytics, and hosts the dashboards and APIs through which users and other systems interact with the deployment.
Application Layer	The end-user-facing software – dashboards, mobile apps, automated alerts – through which the value of the deployment is actually realised.

Table 3: Core components of an IoT system.

2.1 Enabling Technologies

A number of technology families had to mature before IoT at scale became practical. These are grouped here by role, and are each expanded on in later modules of this course:

- **Identification technologies:** RFID and NFC allow objects to carry a machine-readable identity, and GPS allows an object to report its own location – both are prerequisites for treating a physical object as an addressable network entity.
- **Communication technologies:** Short-range protocols (Bluetooth/BLE, Zigbee, Wi-Fi) and long-range, low-power protocols (LoRaWAN, NB-IoT, LTE-M) allow devices with very different power and range requirements to connect (Unit II).
- **Computation platforms:** Inexpensive microcontrollers and single-board computers (Arduino, ESP32, Raspberry Pi) put enough processing power at the edge to run local logic rather than relying on the cloud for every decision (Unit III).
- **Cloud and edge computing:** Elastic cloud storage and compute, combined with edge computing close to the device, together provide the processing capacity a sensor-generated data stream requires (Unit III).
- **Data analytics and AI:** Machine learning applied to sensor data enables anomaly detection and predictive maintenance, turning raw readings into actionable insight (Unit IV).

3 IoT Reference Architecture

To manage the complexity of heterogeneous devices, networks, and applications, IoT systems are almost always described using a **layered reference architecture**. Two forms are in common use: a simpler **three-layer** model, useful for introducing the core data flow, and a more detailed **five-layer** model, which separates out processing and business logic more explicitly. Figure 1 shows both, and how the five-layer model refines the three-layer model.

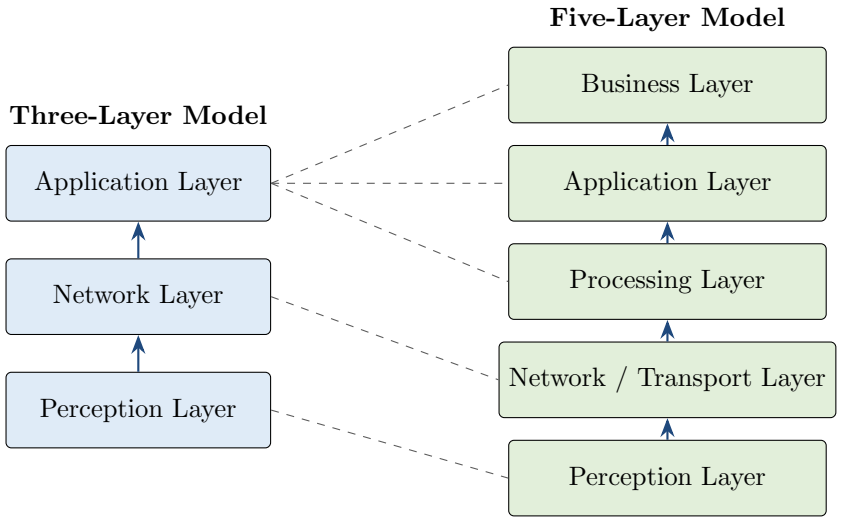


Figure 1: The three-layer and five-layer IoT reference architectures. The five-layer model splits the three-layer Application Layer into Processing, Application, and Business layers, giving finer-grained separation between data handling and business logic.

3.1 The Three Layers

- **Perception Layer:** The physical layer, comprising sensors and actuators that perceive the environment and, where applicable, act upon it. This layer is responsible for data collection at the point of sensing.
- **Network Layer:** Responsible for transmitting data collected by the perception layer to processing systems, using the short- and long-range communication technologies and gateways introduced in Section 2.
- **Application Layer:** Delivers application-specific services to the end user – a dashboard, an alert, or an automated action – built on top of the data the lower two layers have collected and transported.

3.2 The Five Layers

The five-layer model retains the Perception and Network layers but decomposes the three-layer model's Application layer into three more specific layers:

- **Processing Layer** (sometimes called the *middleware* layer): stores, analyses, and pre-processes the incoming data stream – this is where much of the edge and cloud computing discussed in Unit III actually occurs.
- **Application Layer:** Provides application-specific management, delivering processed information to end-user services such as smart-home control or health monitoring.
- **Business Layer:** Sits above the application layer and is concerned with the broader business model – analysing collected data to inform business decisions, manage overall system operation, and evaluate the success of a deployment, rather than with any single sensor reading.

3.3 Edge and Fog Computing

A recurring theme across both models is *where* processing actually happens. **Edge computing** performs computation directly on, or very near, the device that generated the data (for example, running an anomaly-detection model on the microcontroller itself), minimising latency and bandwidth use. **Fog computing** is an intermediate tier, typically implemented on a local gateway or on-premises server, that aggregates and processes data from many edge devices before a smaller, filtered volume of data is sent to the cloud. Both stand in contrast to a purely cloud-centric design, where every raw reading is transmitted for central processing – a design covered in detail in Unit III.

Worked Example

Mapping an application to the five-layer model. A smart irrigation deployment uses soil-moisture sensors (perception), an ESP32 gateway sending data over Wi-Fi (network), a cloud service that filters and stores readings (processing), a mobile app showing current soil moisture (application), and a monthly report used by a farm manager to decide whether to expand the irrigation system (business). Identify which layer each component belongs to.
Solution. Soil-moisture sensors → Perception Layer. ESP32/Wi-Fi transmission → Network Layer. Cloud filtering and storage → Processing Layer. Mobile app display → Application Layer. Monthly expansion decision-making → Business Layer.

4 IoT Devices

Section 2 introduced IoT components at a system level; this section looks more closely at the device types that occupy the Perception Layer and the local computation immediately above it.

4.1 Sensors and Actuators

A **sensor** converts a physical quantity into a measurable electrical signal: common examples include temperature and humidity sensors (e.g., DHT11/DHT22), light-dependent resistors (LDRs) for light intensity, ultrasonic sensors (e.g., HC-SR04) for distance, and passive infrared (PIR) sensors for motion. An **actuator** performs the reverse operation, converting an electrical/digital command into physical motion or change – a relay switching a pump on, a servo motor opening a vent, or an LED indicator changing state. Together, sensors and actuators form the two-way interface between an IoT system and the physical world.

4.2 Microcontrollers and Embedded Systems

A **microcontroller** is a small, self-contained computer – typically combining a processor core, memory, and input/output pins on a single chip – that reads sensors, executes control logic, and drives actuators. An **embedded system** is the broader category of a computing system built into a device to perform a small number of dedicated functions, of which a microcontroller-based IoT node is one common example. Popular platforms used throughout this course's laboratory exercises (Unit III) include Arduino boards, the Wi-Fi-and-Bluetooth-capable ESP32, and the more general-purpose Raspberry Pi, which runs a full Linux operating system rather than executing a single fixed program.

4.3 Identification and Location Technologies

- **RFID (Radio-Frequency Identification):** Uses a reader to wirelessly read a unique identifier stored on a tag, without requiring line of sight or a power source on passive tags – widely used for asset tracking and access control.
- **NFC (Near-Field Communication):** A short-range variant of RFID, typically operating over a few centimetres, used in contactless payment and device-pairing applications.
- **GPS (Global Positioning System):** Provides a device with its own geographic location, enabling applications such as fleet tracking and geofencing.

Worked Example

Battery life of a sensor node. A battery-powered soil-moisture sensor node draws 80 mA while transmitting (for 2 seconds per reading) and 0.5 mA while idle between readings. It takes one reading every 10 minutes and is powered by a 2000 mAh battery. Estimate the battery life, assuming idle current dominates between readings.
Solution. Each 10-minute (600 s) cycle consists of 2 s active at 80 mA and 598 s idle at 0.5 mA. Average current per cycle:

$$I_{\text{avg}} = \frac{(2\,\text{s})(80\,\text{mA}) + (598\,\text{s})(0.5\,\text{mA})}{600\,\text{s}} = \frac{160 + 299}{600} \approx 0.765\,\text{mA}.$$

Battery life $\approx \dfrac{2000\,\text{mAh}}{0.765\,\text{mA}} \approx 2614$ hours, or roughly 109 days – illustrating why minimising the

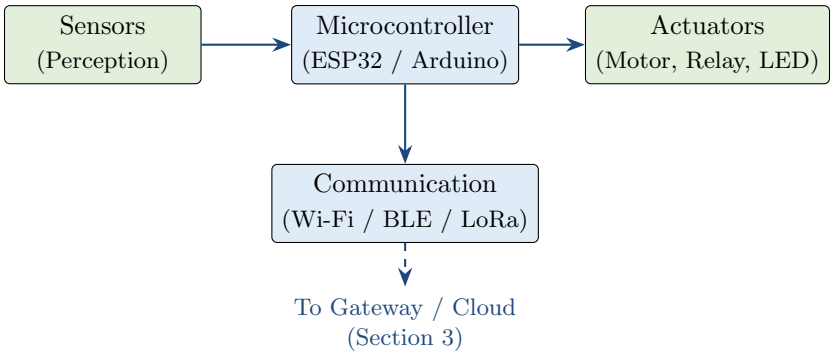


Figure 2: Anatomy of a single IoT device node. The microcontroller reads sensors, drives actuators, and manages a communication module that links the node to a gateway or directly to the cloud.

fraction of time spent transmitting is central to long-lived battery-powered IoT design.

5 The IoT Data Journey

Ultimately, every layer discussed above exists to move data from a physical measurement to a decision or action. It is useful to think of this as a five-stage pipeline, shown in Figure 3, that recurs across every application domain covered in Unit V.



Figure 3: The IoT data journey: from raw sensing through to the dashboards that display real-time monitoring information to end users.

- **Data acquisition:** A sensor samples a physical quantity at some rate, producing a raw digital reading – the sampling rate itself is an important design parameter, trading data fidelity against power and bandwidth cost.
- **Data transmission:** Readings are moved from the device, over the network layer's chosen protocol, toward a gateway or directly to the cloud.
- **Data storage:** Time-stamped readings are persisted, typically in a time-series database optimised for large volumes of sequential, timestamped records.
- **Data analytics:** Stored data is processed to extract insight – from simple threshold alerts to machine-learning-based anomaly detection and predictive maintenance (Unit IV).
- **Real-time monitoring / dashboards:** The processed results are presented to a human operator, or fed back automatically to an actuator, closing the loop between sensing and action.

Worked Example

Estimating daily data volume. A smart-city deployment has 500 air-quality sensors, each sending a 200-byte reading once every 5 minutes. Estimate the total data volume generated per day.

Solution. Readings per sensor per day $= \dfrac{24 \times 60}{5} = 288$. Data per sensor per day $= 288 \times 200\,\text{B} = 57{,}600\,\text{B} \approx 56.25\,\text{KB}$. Total for 500 sensors $\approx 500 \times 56.25\,\text{KB} = 28{,}125\,\text{KB} \approx 27.5\,\text{MB/day}$ – a modest volume for a single small deployment, but one that scales linearly with sensor count and motivates the edge-filtering strategies discussed in Unit III as deployments grow to thousands of devices.

IoT in Practice: Connecting Back to the Course

The layered reference architecture, device taxonomy, and data pipeline introduced in this module are not abstract for their own sake: every application in Unit V – smart agriculture's soil-moisture-to-irrigation loop, healthcare's wearable-to-dashboard monitoring, or a smart city's traffic sensors feeding a control centre – is simply a specific instantiation of the same Perception → Network → Processing → Application pipeline built from the same sensor, microcontroller, gateway, and cloud building blocks. Unit II supplies the communication technologies that implement the Network Layer; Unit III supplies the hardware and cloud/edge platforms that implement the Processing Layer; Unit IV secures this pipeline and extracts analytical value from it; and Unit V shows the complete pipeline assembled end to end for seven real-world domains.

6 Solved Problems

Worked Example

Problem 1. A factory installs vibration sensors on ten machines. Each sensor sends a reading directly to one fixed on-site server using a proprietary serial protocol, with no other system able to access the data. Is this deployment better classified as IoT or classic M2M, and why?
Solution. Classic M2M: the communication is point-to-point, uses a proprietary protocol rather than shared Internet infrastructure, and the data is not exposed to any broader network of services – exactly the pattern distinguished from IoT in Section 1.3.

Worked Example

Problem 2. Place the following into the correct layer of the five-layer IoT reference architecture: (a) a temperature sensor, (b) an MQTT message travelling over Wi-Fi, (c) a cloud function that computes a daily average temperature, (d) a mobile app showing the current temperature, (e) a quarterly report used to decide whether to install more sensors.
Solution. (a) Perception Layer; (b) Network/Transport Layer; (c) Processing Layer; (d) Application Layer; (e) Business Layer.

Worked Example

Problem 3. A wearable heart-rate monitor samples once per second, with each reading occupying 4 bytes, and transmits a batch of readings once every minute. Compute (a) the size of one transmitted batch, and (b) the total data generated per patient per 24-hour day.
Solution. (a) 60 readings/minute × 4 bytes = 240 bytes per batch. (b) $24 \times 60 = 1440$ batches/day × 240 bytes = 345,600 bytes ≈ 337.5 KB per patient per day.

Worked Example

Problem 4. Explain, in one or two sentences, why a purely cloud-centric architecture (sending every raw reading to the cloud for all processing) becomes increasingly impractical as an IoT deployment grows from tens to millions of devices, and name the two architectural concepts introduced in Section 3.4 that address this.

Solution. Bandwidth, latency, and cloud storage/processing costs all scale with the number of devices and their sampling rates, so routing every raw reading to a distant data centre becomes slow and expensive at scale; edge computing and fog computing address this by processing data closer to where it is generated, sending only filtered or summarised results onward.

7 Summary

IoT extends network connectivity from computers to physical objects, distinguished from plain Internet connectivity by its heterogeneity and scale, from M2M by its use of shared Internet-based infrastructure rather than fixed point-to-point links, and from cyber-physical systems by not requiring tight real-time feedback control. Every IoT system is built from a recurring set of components – sensors, actuators, microcontrollers, connectivity, gateways, and cloud platforms – organised into either a three-layer (Perception, Network, Application) or a more detailed five-layer (Perception, Network, Processing, Application, Business) reference architecture, with edge and fog computing determining where in this stack processing actually takes place. Data moves through the system via a consistent five-stage pipeline – acquisition, transmission, storage, analytics, and monitoring – that recurs, in specialised form, across every application domain covered later in this course.

References

1. Course syllabus, *Internet of Things (IoT) with Applications* — Unit I: Introduction to IoT.
2. A. Al-Fuqaha, M. Guizani, M. Mohammadi, M. Aledhari, and M. Ayyash, "Internet of Things: A Survey on Enabling Technologies, Protocols, and Applications," *IEEE Communications Surveys & Tutorials*, 2015.
3. J. Gubbi, R. Buyya, S. Marusic, and M. Palaniswami, "Internet of Things (IoT): A Vision, Architectural Elements, and Future Directions," *Future Generation Computer Systems*, 2013.
4. Course notes, *Internet of Things (IoT) with Applications* – Module 1, Introduction to IoT.